# Validating data: what to keep, and what the model already knows

*Targeted selection, judges and humans, membership tests, and the discipline that keeps a good benchmark score from being a lie*

---

Chapter 10 ended with a dataset you can trust in one specific sense: you know where every document came from, you removed the duplicates and the junk, and you wrote down the recipe. It said nothing about whether the dataset is the *right* one. Here is the situation that exposes the difference. Your 7B model, trained on 2T tokens of a FineWeb-Edu–style corpus, scores 62 on MMLU. A competitor's, same size, same token count, scores 66. Three explanations are on the table and they call for opposite actions: their data was closer to what the benchmarks measure (then you should select differently), their data was better in some way your classifier does not see (then you should audit quality), or their training set contained the benchmark (then their number is worth nothing and you should stop chasing it). You cannot tell the three apart by looking at the leaderboard. You can tell them apart with about a day of CPU work and a small reference model, which is what this chapter is about: three questions — does the data match the target? is it good? does the model already know it? — each with a measurement, a cost, and a trap.

The chapter is the hinge between the pre-training pipeline (Chapter 10) and the post-training data you will build or buy later (Chapters 23 and 27), because the three questions are the same on both sides; only the units change — documents for pre-training, prompt–response pairs for SFT, preference pairs for alignment. The running examples are therefore mixed: FineWeb-Edu sample-10BT for the pre-training side; UltraChat-200k, UltraFeedback, and OpenThoughts3 for the post-training side.

## 11.1 Three questions, three costs

Start with what "validating" a dataset could even mean. A dataset is a sample from some distribution; training on it moves the model toward that distribution; you care about performance on a *different* distribution — the tasks users bring — which you approximate with benchmarks. So a dataset is good exactly when training on it helps on the target, and the only ground-truth test is to train and evaluate, which is the test you cannot afford to run per candidate dataset at full scale. Everything in this chapter is a proxy for that test, and the proxies sort into three questions that are logically separate and need separate tools.

**Does it match the target?** Relevance is a property of the pair (data, task). A corpus can be pristine and useless for your users; a messy forum dump can be the single most valuable source for a coding model. The tools here compare documents to a description of the target — benchmark examples, a few-shot set, a gradient — and rank them (§11.2). **Is it good?** Quality is closer to a property of the document alone: coherent, informative, correct, not boilerplate. The tools are judges, model-based or human, and the engineering is in calibrating them (§11.3). **Does the model already know it?** This is the question most teams skip, and it has two faces. Benign: an SFT example the base model already answers correctly contributes nothing but a gradient toward what it already does, so measuring what the model knows tells you where the *marginal* value of data is (§11.4). Malign: a benchmark item that was in the training set makes the evaluation a memory test, and the same measurement that finds marginal value finds contamination (§11.5).

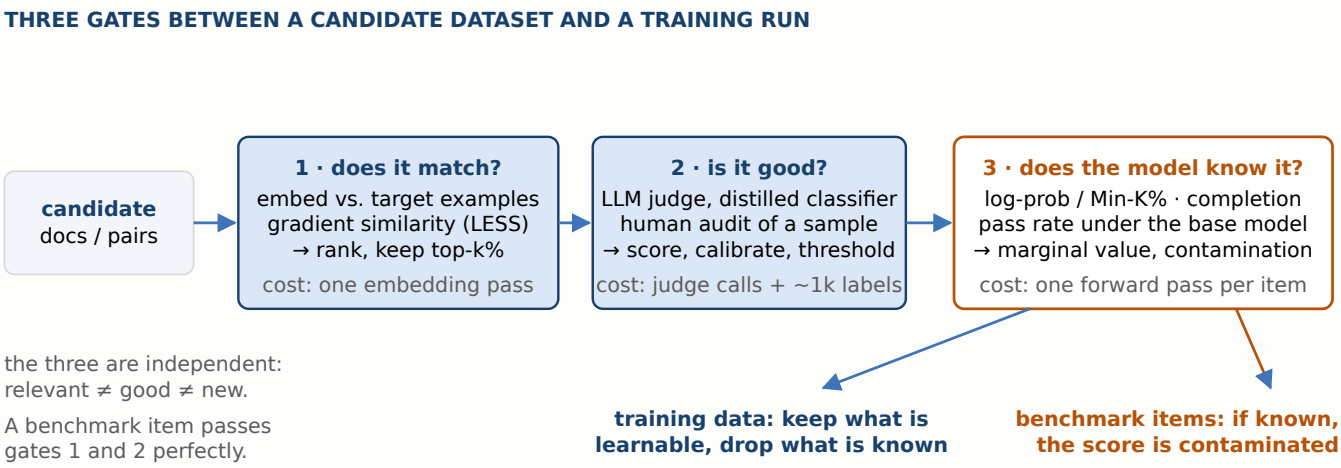



**Figure 11.1 — Relevant, good, and new are three different properties.** The third gate is the one that is usually missing, and it is the only one that distinguishes a benchmark score you earned from one you memorized: a leaked test item is maximally relevant and perfectly good.

The order matters for cost. Relevance ranking is one embedding pass over a sample and a cheap classifier for the rest; quality judging costs a model call per document and is run only on what survived; the knowledge test is a forward pass of your own model per item and is run on everything you are about to evaluate on. Keep the three separate in your lineage record too — a document carries a relevance rank, a quality score, and a membership score — because the decisions they feed are different, and because the last point of Figure 11.1 is the one to internalize: a leaked test item is maximally relevant and perfectly good. Nothing in gates 1 and 2 will ever catch it.

## 11.2 Does it match the target? Targeted selection

The quality classifiers of Chapter 10 ask an LLM what "educational" means and keep documents that score high. That is a definition of good data written by a prompt engineer. The 2025 result that changed how labs think about selection asks a sharper question: what if "good" simply means *close to the tasks you will be evaluated on*? Apple's **BETR** (Benchmark-Targeted Ranking) tested that directly, across more than 500 models from $10^{19}$ to $10^{22}$ FLOPs, and the answer is that matching data to the target beats every hand-written notion of quality: a 2.1× compute multiplier over DCLM-Baseline and 4.7× over unfiltered data, with the targeted set winning on 9 of 10 core benchmarks[1]. The mechanism is simple enough to reimplement in an afternoon, and its failure modes are the most instructive part.

## The BETR mechanism

Take the *training* splits of your benchmarks — BETR used over 14,000 examples from dozens of tasks — and embed each example individually with a sentence-embedding model (Arctic-Embed L 2.0 in the paper; any strong open embedder will do). Embed a random sample of about 10 million pre-training documents, under 0.1% of the pool. Now let every benchmark example rank every sampled document by cosine similarity, and give each document the *best* rank it achieves across all examples:

$$s(x) \;=\; \max_{j} \operatorname{rank}_j(x),$$

where $\operatorname{rank}_j(x)$ is the position of $x$ when all sampled documents are sorted by $\cos\big(e(x), e(q_j)\big)$, "max" means best position, $e(\cdot)$ is the embedder, and $q_j$ the $j$-th benchmark example. Individual examples work better than per-benchmark centroids or clusters, because a centroid of MMLU is a vague point in embedding space and a single MMLU question about organic chemistry is a sharp one[1]. The ranked sample is then turned into a cheap classifier exactly as in Chapter 10: label the top 10% positive and the rest negative, train fastText, score the full pool, keep the top fraction of tokens. Two steps — embed-and-rank on a sample, then classify everything — and the two-step design matters: training the classifier directly to separate benchmark examples from web text scored 59.7 on the core average versus 62.2 for the ranked version, because benchmark examples look like benchmark examples (short, question-shaped) and the classifier learns the format, not the content[1].

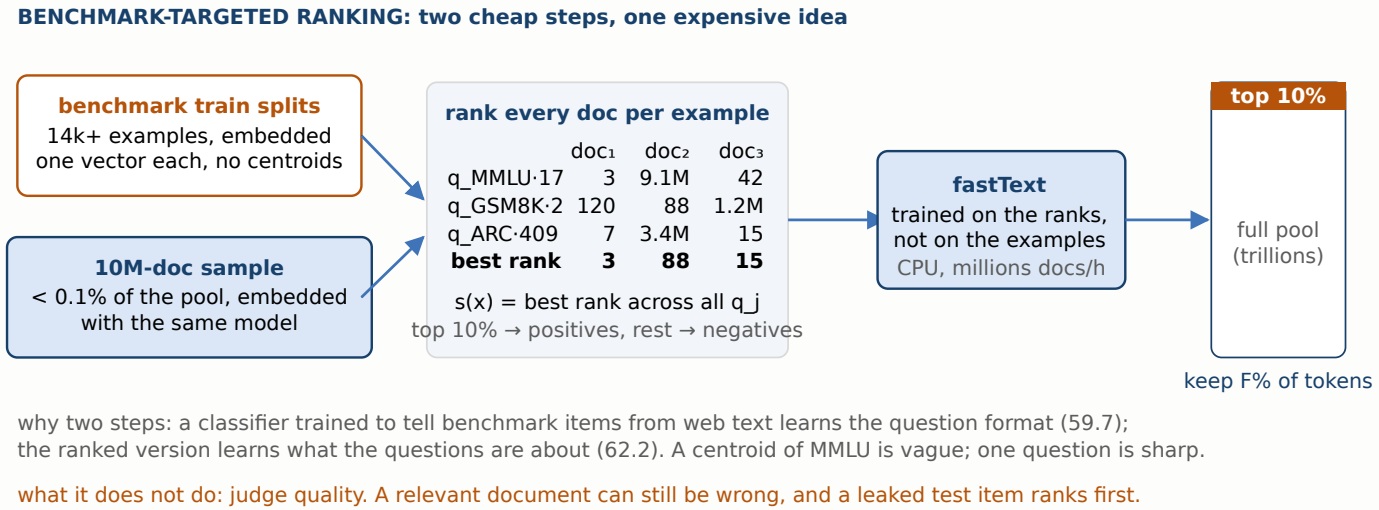



**Figure 11.2 — BETR in one picture.** The expensive idea is "relevance to the evaluation" as the definition of good data; the two cheap steps make it run on a trillion-token pool in CPU time. Numbers are the paper's core-benchmark averages at 7B-1x[1].

## How much to keep depends on how big you are

BETR's most useful finding for a team with a fixed budget is the one about the *fraction*. Across their 53-model scaling sweeps the optimal retained fraction of tokens grows with compute as

$$F_{\text{opt}}(C) \;\approx\; 4 \times 10^{-5}\, C^{0.25},$$

with $F$ in percent and $C$ in training FLOPs: the top 3% is best below about $10^{21}$ FLOPs, the top 10% around $10^{22}$, and the top 30% beyond $10^{23}$[1]. Read it as a statement about capacity: a small model cannot absorb the long tail of a broad corpus, so you should hand it only the most on-target slice; a large model has room for breadth and is hurt by a filter that is too tight. For the 7B/2T budget of our running example, $C = 6ND \approx 8.4 \times 10^{22}$, the formula says keep roughly the top 20% — and it also says that the dataset decision is *not* separable from the compute decision, which is why the data chapter had to come after the scaling-laws chapter. Figure 11.3 plots the law with the three measured points.

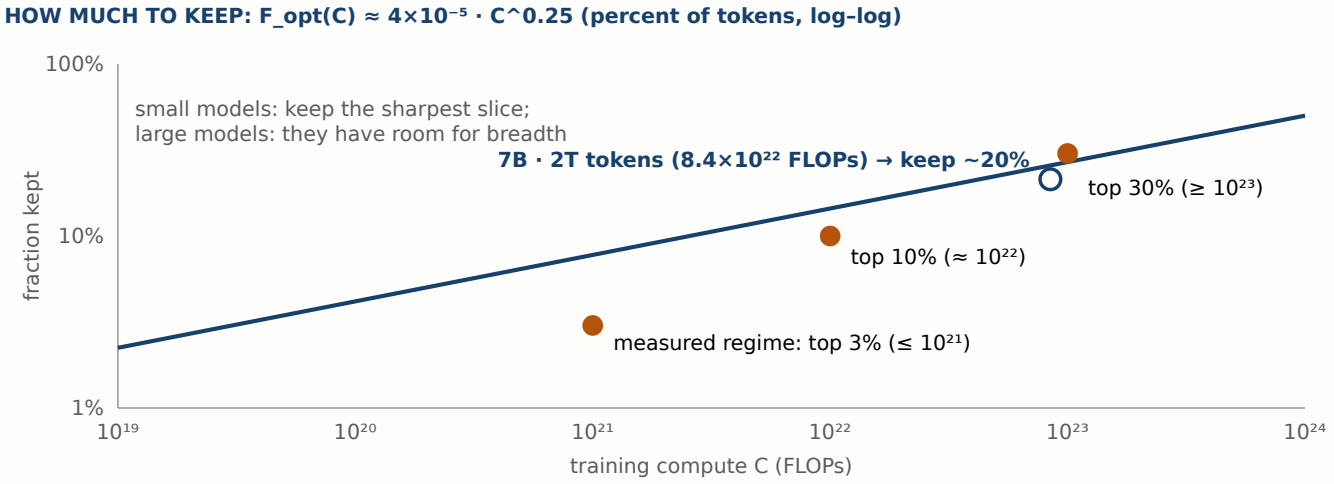



**Figure 11.3 — The filter should loosen as the model grows.** The line is BETR's fitted law; the dots are the three regimes its sweeps identified (the fit smooths them); the open circle is this book's 7B/2T running example. Not far above $10^{24}$ FLOPs the law passes 100% — the regime where you need more data, not less, and Chapter 27's synthesis takes over.

## The failure mode is the method working

Everything that makes BETR effective also makes it dangerous, and the paper is unusually honest about it. The targeted set, trained for the 10 core benchmarks, ranks *third* on 39 held-out benchmarks it was not targeted at — 44.2 versus 46.3 for plain DCLM-Baseline — while a set targeted at those 39 instead improves the core average by only 0.2 points[1]. In other words, selecting data by similarity to a benchmark makes the benchmark stop measuring general capability, which is Goodhart's law with an embedding model. Two more negative results sharpen the picture: targeting a domain by name (GitHub) scored *below* no filtering at all, 52.3 versus 56.0, because "looks like GitHub" is not the same as "teaches what GitHub teaches"; and 13-gram decontamination made almost no difference (62.5 without, 62.3 with), which tells you the gains are from distribution matching and not from leaked items — but also that the method would happily keep leaked items if they were there[1]. The practical rules follow directly. Target with the training splits of a *broad* suite, never a single benchmark (single-benchmark targeting gave core averages of 53.5–58.7 against 62.3 for the joint target). Hold out a second suite you never target and report it next to the first; if the gap between the two grows as you tighten the filter, the filter is the problem. And run the contamination test of §11.5 on the *selected* set, because selection by relevance is a magnet for leaks.

## Gradients instead of embeddings: selection for post-training

For instruction data the same idea has a sharper tool. Instead of asking whether an example *looks like* the target, ask whether a gradient step on it would *reduce the loss* on the target. First-order influence makes that computable: for a training example $z$ and a target example $z'$, the change in target loss from one SGD step on $z$ is approximately $-\eta\,\nabla\ell(z')^{\top}\nabla\ell(z)$, so examples whose gradients align with the target's gradient are the ones to keep. The gradient of a 7B model is 7 billion numbers per example, which is where **LESS** earns its name: it projects LoRA gradients to a few thousand dimensions with a random projection (which preserves inner products in expectation), stores them once in a reusable datastore, corrects the formula for Adam's preconditioning, and scores each example by cosine similarity to the few-shot target set:

$$\mathrm{Inf}(z, z') \;\propto\; \frac{\langle \Pi\,\tilde{\nabla}\ell(z),\ \Pi\,\tilde{\nabla}\ell(z')\rangle}{\|\Pi\,\tilde{\nabla}\ell(z)\|\ \|\Pi\,\tilde{\nabla}\ell(z')\|},$$

where $\Pi$ is the random projection and $\tilde{\nabla}$ the Adam-adjusted gradient. Training on the LESS-selected 5% of a 270k-example instruction mix often outperformed training on the full mix, and — the same lesson as the 125M perplexity reference in Chapter 10 and the 280M DoReMi proxy — the gradients of a small model selected good data for larger models of a different family[2]. The cost is one forward–backward pass per candidate with LoRA, which is affordable for a 200k-example SFT set and not for a trillion-token crawl; that is the split in practice: embeddings for pre-training, gradients for post-training.

```
# BETR-lite on FineWeb-Edu sample-10BT: rank docs by best match to benchmark train
items
import numpy as np, faiss
from sentence_transformers import SentenceTransformer
emb = SentenceTransformer("Snowflake/snowflake-arctic-embed-l-v2.0")
Q = emb.encode(target_examples, normalize_embeddings=True)  # train splits of a
BROAD suite
D = emb.encode(doc_sample, normalize_embeddings=True)        # ~10M docs in
production
index = faiss.IndexFlatIP(D.shape[1]); index.add(D)
best_rank = np.full(len(D), len(D))                           # worst possible
rank
for q in Q:  # each example ranks all docs
    _, order = index.search(q[None], len(D))                  # descending cosine
    best_rank[order[0]] = np.minimum(best_rank[order[0]], np.arange(len(D)))
keep = best_rank < 0.10 * len(D)  # top-10% positives for fastText
# fastText(keep vs. rest) → score the full pool → keep F_opt(C) percent of tokens
# then run §11.5 on the kept set: relevance ranking attracts leaked test items
```

## 11.3 Is it good? Judges, classifiers, and the humans who calibrate them

Relevance ranking will happily keep a confidently wrong physics explanation because it is about physics. Quality is the orthogonal axis, and since 2024 the standard instrument for it is a language model asked to grade. The pattern has three tiers, and the mistake teams make is to skip the third. **Tier one** is the frontier judge: a strong instruct model reads a document (or a prompt–response pair) with a rubric and returns a score; FineWeb-Edu's rubric asked Llama-3-70B-

Instruct for a 0–5 "educational value" and the DCLM pipeline asked for "is this instruction-like"[3]. **Tier two** is distillation of the judge into a classifier, exactly as in Chapter 10 — 450k judged documents, an embedding-plus-regressor that scores a trillion tokens on CPUs — because the judge costs about a thousand times more per document than the classifier and the two agree closely enough (F1 0.82 at the threshold FineWeb-Edu used) that running the judge on everything would buy nothing[3]. The same two tiers power Ask-LLM for pre-training, where rejecting 90% of the data on the judge's say-so still beat training on all of it and converged up to 70% faster[4]. **Tier three** is the part that makes the first two trustworthy: a human-labeled sample, drawn at random from the *distribution the classifier will be applied to*, against which both judge and classifier are scored.

## What a judge gets wrong, and how to measure it

LLM judges are good — the original MT-Bench study found GPT-4's agreement with human preferences above 80%, the same level at which humans agree with each other[5] — but their errors are systematic rather than random, which is worse for a filter than noise would be. The documented biases are worth memorizing because they turn into dataset artifacts. *Position bias*: in pairwise comparisons the judge prefers whichever response it saw first, which is why every serious pipeline judges both orders and keeps only consistent verdicts. *Verbosity bias*: longer responses score higher at equal content, so a quality-filtered SFT set drifts long, and the model trained on it learns to pad. *Self-preference*: a judge rates text in its own style above text of equal quality in another style, so a corpus judged by one model family converges on that family's voice — a real problem for a Spanish-first model being graded by an English-centric judge, which tends to score fluent Latin American register below textbook Castilian. And *rubric literalism*: "educational" as a rubric word pulls toward encyclopedic prose and away from the forum thread where someone actually explains why the compiler error happens, which the 2025 selection results say is often the more valuable document[1].

The measurement that catches all four is the same: a calibration set. Draw $n$ items uniformly at random from the candidate pool, have two or more humans label them with the same rubric, and compute three numbers. Inter-annotator agreement, with Cohen's $\kappa$, tells you whether the rubric is even well-defined:

$$\kappa \;=\; \frac{p_o - p_e}{1 - p_e},$$

where $p_o$ is the observed fraction of items the two annotators agreed on and $p_e$ the agreement expected by chance from their marginal label rates; a $\kappa$ below about 0.6 means the humans do not agree with each other and no judge can be validated against them — fix the rubric first. Judge–human agreement, the same $\kappa$ between the judge's labels and the humans' majority, tells you how much of the judge to trust. And the *confusion matrix at the threshold you will deploy* tells you what the filter will actually do: a classifier with 0.82 F1 at threshold 3 is a different filter from the same classifier at threshold 2, and the false-positive population (junk that passes) is what ends up in your model.

How big should $n$ be? For a binary label with true positive rate $p$, the standard error of an estimate from $n$ items is $\sqrt{p(1-p)/n}$, so the sample needed for a margin $e$ at confidence $z$ is

$$n \;=\; \frac{z^2\, p\,(1-p)}{e^2} \;\approx\; \frac{1.96^2 \times 0.25}{0.03^2} \;\approx\; 1{,}070 \quad \text{for } \pm 3\% \text{ at } 95\%,$$

taking $p = 0.5$ as the worst case. A thousand labels is one or two days for two annotators and is the cheapest insurance in the pipeline; stratify it by source and by language so that the estimate holds per stratum, not only in aggregate. Figure

11.4 shows how the sample size curve bends: going from ±3% to ±1% costs nine times the labels, which is rarely worth it; going from ±10% to ±3% costs eleven times and almost always is.

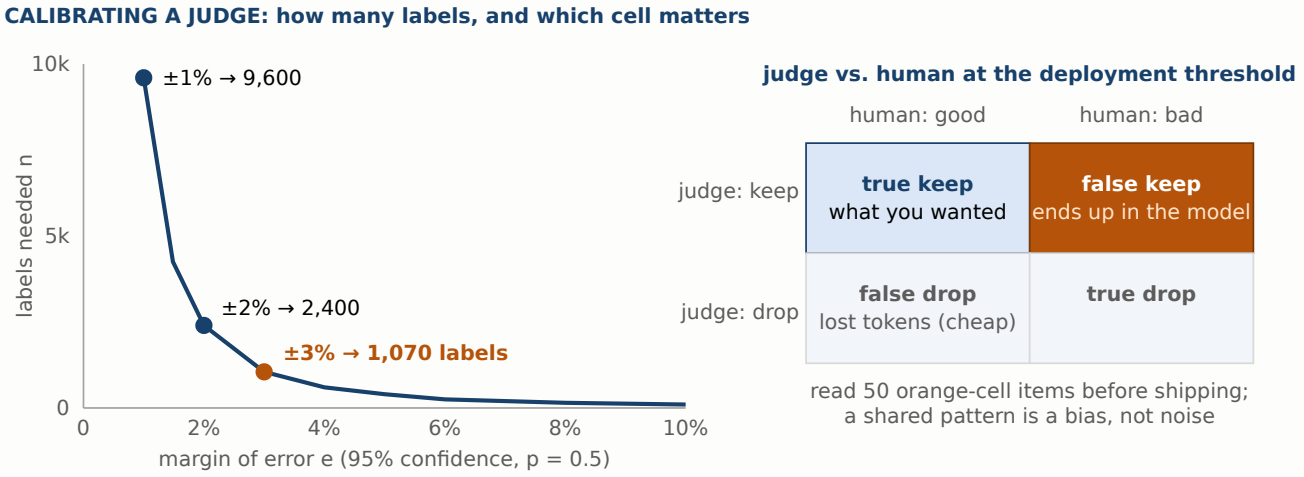



**Figure 11.4 — A thousand labels is the price of trusting a judge.** Left: the sample-size curve from the margin-of-error formula. Right: the cell to read by hand is the false-keep cell, because a judge's errors are correlated and what it wrongly passes becomes a systematic ingredient of the model.

```
# calibrate a judge against a stratified human sample (two annotators, binary
rubric)
import numpy as np
from sklearn.metrics import cohen_kappa_score, confusion_matrix
sample = stratified_sample(pool, n=1100, by=["source", "lang"])  # uniform within
strata
h1, h2 = labels_from(annotator_1), labels_from(annotator_2)      # 0/1 per item
print("human–human κ:", cohen_kappa_score(h1, h2))  # < 0.6 → fix the rubric first
gold = adjudicated_majority(h1, h2)  # adjudicate the ~10% disagreements
score = classifier.predict_proba([x.text for x in sample])[:, 1] # the distilled
tier-2 model
for thr in (0.5, 0.7, 0.9):
    pred = (score >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(gold, pred).ravel()
    print(thr, "κ=%.2f precision=%.2f recall=%.2f false-keep=%d" % (
        cohen_kappa_score(gold, pred), tp / (tp + fp), tp / (tp + fn), fp))
# then: read 50 false-keep items; log the threshold and this table in the dataset
card
```

## 11.4 Does the model already know it? Measuring marginal value

Here is the question that separates data engineering from data science. Suppose the SFT set you are about to use is relevant and good by every test above. If the base model, prompted with the instruction, already produces the reference answer, what does training on that pair teach it? Approximately nothing — a gradient toward what it already does, with

the small cost of making it more confident. The value of a training example is its *marginal* value given what the model knows, and that is measurable because the model will tell you how surprised it is. Three instruments, in order of cost.

## Log-probability and the Min-K% test

The cheapest signal is the one from Chapter 10's curriculum section, now pointed at the model you are about to train rather than a reference: the per-token log-probabilities $\log p_\theta(x_t \mid x_{<t})$ of the example under the current checkpoint. Their mean is the negative log-perplexity, and a very high mean — the model predicts the text nearly token for token — means either the content is trivial or the model has seen it. Those two cases are the point of the **Min-K% Prob** test. Text the model has never seen almost always contains a few tokens it finds very surprising (a rare name, an unexpected number), while text it was trained on has had those outliers pushed up; so instead of the mean over all tokens, average the $k\%$ *lowest* token log-probabilities:

$$\mathrm{MinK}(x) \;=\; \frac{1}{|S_k|}\sum_{t \in S_k} \log p_\theta(x_t \mid x_{<t}), \qquad S_k = \text{the } k\% \text{ of positions with the smallest } \log p_\theta,$$

and flag $x$ as "seen" when the score exceeds a threshold set on known-unseen text ($k$ around 20 works across model sizes). On WikiMIA — Wikipedia text dated before and after a model's training cutoff, so that membership is known — it improved over the previous detection methods by 7.4 points, needs no reference model and no access to the training corpus, and transfers directly to the two uses this chapter cares about: detecting benchmark items in a model's training data, and detecting copyrighted books[6]. Figure 11.5 shows the intuition as two token-level profiles. For data validation the reading is: run Min-K% over every candidate example, and treat the top tail — the examples with no surprising tokens at all — as already known, to be down-weighted or dropped, after checking a sample by hand (some of them are simply trivial).

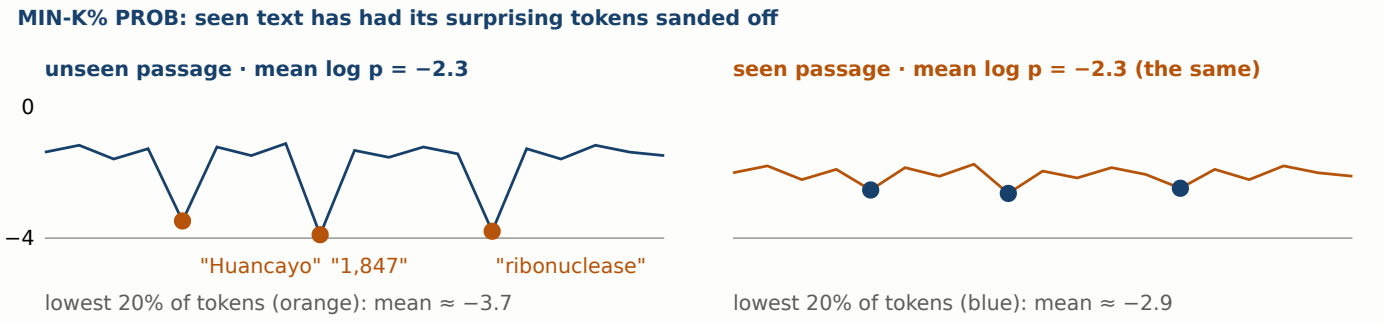



**Figure 11.5 — Membership is in the tail, not the mean.** Two passages with identical average log-probability; the one the model trained on has lost its surprising tokens. The same test serves both faces of the third question — contamination of a benchmark and marginal value of a training example.

## Instruction-following difficulty

For prompt–response pairs the better instrument conditions on the prompt. The **IFD** score asks how much the instruction *helps* the model produce the response, by comparing the response's loss with and without the instruction:

$$\mathrm{IFD}(Q, A) \;=\; \frac{\ell_\theta(A \mid Q)}{\ell_\theta(A)}, \qquad \ell_\theta(A \mid Q) = -\frac{1}{|A|}\sum_t \log p_\theta(a_t \mid Q, a_{<t}),$$

where $\ell_\theta(A)$ is the same mean loss on the answer alone. A ratio near 1 means the instruction did nothing — the model would have written that answer anyway, or the answer is generic enough that no instruction is needed — and a ratio near 0 means the answer is trivially determined by the prompt (copying, formatting). The informative examples sit in between and toward the high side: answers the model finds hard *even given* the instruction. Training on the top 5% of Alpaca and 10% of WizardLM by IFD matched or beat training on the full sets, which, together with Deita's result that 6,000 examples selected for complexity, quality, and diversity matched models trained on ten times more[7][8], is the quantitative version of a lesson the field learned the hard way: for SFT, the dataset you want is small, hard, and new to the model.

## Pass rate: the test for reasoning data

For prompts with a checkable answer — math, code, anything with a verifier — the cleanest marginal-value measurement is to *sample*. Draw $n$ responses from the current model at temperature 1, verify each, and record the pass rate $\hat{p}$. A prompt with $\hat{p} = 1$ is known; a prompt with $\hat{p} = 0$ is, for now, out of reach; the ones that teach are in between, and in reinforcement learning this is not a heuristic but the gradient itself: the GRPO advantage (Chapter 25) is zero whenever all $n$ samples share the same reward, so prompts at $\hat{p} \in \{0, 1\}$ contribute nothing and dynamic sampling simply discards them[9]. The expected magnitude of the learning signal scales with the variance of the outcome,

$$\mathrm{Var}(\text{reward}) = \hat{p}\,(1 - \hat{p}),$$

maximal at $\hat{p} = 0.5$, and the practical recipe is to bin prompts by $\hat{p}$, spend the budget on the middle bins, and re-measure every few hundred steps because the bins drift as the model learns — which is a curriculum, defined by the model's own competence rather than by a static difficulty score (the end point of the road Chapter 10's §10.6 started down). The same bins, measured on the *base* model before any training, are the sampling audit for a reasoning dataset: if 80% of OpenThoughts-style prompts are already solved by your base model at $\hat{p} > 0.9$, the dataset is three times smaller than it looks.

```
# marginal value of candidate SFT / RL prompts under the current checkpoint
import torch, numpy as np
@torch.no_grad()
def token_logprobs(model, tok, text):
    ids = tok(text, return_tensors="pt").input_ids.cuda()
    lp = torch.log_softmax(model(ids).logits[0, :-1].float(), -1)
    return lp.gather(1, ids[0, 1:, None])[:, 0].cpu().numpy()
def min_k(lp, k=0.2):                       # Min-K% Prob: mean of the lowest k
fraction
    return np.sort(lp)[: max(1, int(k * len(lp)))].mean()
def ifd(model, tok, q, a):                  # instruction-following difficulty
    with_q = -token_logprobs(model, tok, q + a)[-len(tok(a).input_ids):].mean()
```

```
    alone  = -token_logprobs(model, tok, a).mean()
    return with_q / alone  # ≈1: instruction didn't help; high: hard even with it
def pass_rate(model, tok, prompt, verify, n=8):
    outs = sample(model, tok, prompt, n=n, temperature=1.0)
    return np.mean([verify(o) for o in outs])  # RL: keep 0 < p̂ < 1; SFT: p̂ < 0.9
# log all three into the lineage record; re-measure pass rates every few hundred
steps
```

## 11.5 Contamination and benchmaxxing

Now turn the third question around. Everything in §11.4 measured whether the model already knows a *training* example. Apply the same instruments to a *benchmark* item and a "yes" has a different name: contamination. Chapter 9 gave you the defensive ring — 13-gram overlap removal at dataset build time, a private probe set, and a discount on any public number — and this section is about why the ring is not enough and what the stronger tests look like, because the 2024–2025 evidence says the problem is both more common and more subtle than n-gram overlap can see.

The clearest demonstration is **GSM1k**: a thousand grade-school math problems written by humans to match GSM8K's difficulty, style, and answer distribution, never published in training-scrapable form. Evaluated on both, leading models dropped up to 8 points from GSM8K to GSM1k, several families showed systematic overfitting at every size, and — the diagnostic that matters for this chapter — a model's probability of *generating* GSM8K examples verbatim correlated with its GSM8K–GSM1k gap (Spearman $r^2 = 0.36$), direct evidence of partial memorization[10]. Note what 13-gram decontamination would have caught: only items present as literal strings. A paraphrased GSM8K problem, a translated one, or the same problem with the numbers changed passes an n-gram filter and still teaches the model the benchmark's template, which is why BETR's decontamination ablation found nothing to remove while the method was busy selecting the most benchmark-shaped documents on the web[1].

**Benchmaxxing** is the name that stuck in 2025 for optimizing a model toward benchmark numbers by means that do not improve the model: training on test-shaped data, selecting checkpoints on the test, or — the version the leaderboard analysis of Chatbot Arena documented — testing many private variants and publishing only the best, with 27 private variants of one provider found before a single release, and the top two closed providers receiving an estimated 19–20% each of all Arena battle data while 83 open-weight models shared under 30% between them; the authors estimated that even a limited amount of extra Arena data could yield up to 112% relative gains on the Arena distribution[11]. The lesson for a data team is not about ethics, it is about measurement: a benchmark that is also a training target has stopped being a benchmark, and the only numbers you can trust are the ones on distributions you did not optimize for. That is the design behind Tülu 3's split into a *development* suite used to make every data and recipe decision and an *unseen* suite that is never looked at until the end, so that the gap between the two is an honest estimate of how much the development process overfit[12] — and behind this book's Chapter 9 insistence on a private probe.

### Three tests, strongest last

The tests you should run on every benchmark in your suite, against every model you intend to report, form a ladder. The **overlap test** is the Chapter 10 decontamination run in reverse: for each benchmark item, search the training set for shared 13-grams (or 8-grams for short items); it catches verbatim leaks and nothing else. The **membership test** is Min-

K% on each benchmark item under the trained model, with the threshold set on held-out text written after the training cutoff; it catches verbatim and near-verbatim leaks regardless of whether you can see the training set — which is the case for any model you did not train yourself, and for the base model you downloaded. The **completion test** is the strongest and the simplest to explain to a committee: give the model the first half of a benchmark item — the question up to a cut point, or the first two sentences of a passage — and ask it to continue; if it reproduces the second half (measured with ROUGE-L or exact match against the original, compared with its ability to do the same for freshly written items of the same kind), it has seen it. Guided prompting adds the benchmark's name and split to the instruction ("complete this item from the GSM8K test set") and measures how much that raises the match, which isolates memorization of the benchmark specifically from general fluency[13]. Report the three results next to the score, per benchmark, in the dataset card, with the same discipline Chapter 9 asked for noise bars.

```
# completion test: does the model finish benchmark items it should never have
seen?
from rouge_score import rouge_scorer
rs = rouge_scorer.RougeScorer(["rougeL"])
def completion_score(model, tok, items, guided=False, cut=0.5):
    hits = []
    for it in items:
        words = it["text"].split(); k = int(cut * len(words))
        prefix, truth = " ".join(words[:k]), " ".join(words[k:])
        hint = (f"Complete this item from the {it['benchmark']} {it['split']}
split.\n"
                if guided else "")
        out = generate(model, tok, hint + prefix, max_new_tokens=len(words) - k +
8,
                       temperature=0)
        hits.append(rs.score(truth, out)["rougeL"].fmeasure)
    return float(np.mean(hits))
for bench in suite:                          # control: fresh items of the same
kind
    s_bench = completion_score(m, tok, bench.items, guided=True)
    s_ctrl  = completion_score(m, tok, bench.fresh_controls, guided=True)
    print(bench.name, "bench %.2f ctrl %.2f gap %.2f" % (s_bench, s_ctrl, s_bench
- s_ctrl))
# a gap well above the controls' noise is contamination; report it beside the
score
```

## 11.6 Sampling a good post-training set

Pre-training data is validated per document; post-training data is validated per *set*, because the thing that matters — what mix of behaviors the model learns — is a property of the whole. Three public datasets are the running examples here because they are what you would actually start from. **UltraChat-200k** is 200k multi-turn dialogues, GPT-generated, filtered from a 1.5M original; it is the SFT set behind Zephyr and the canonical "teach the model to chat" starting point. **UltraFeedback** is 64k prompts with four model responses each, scored by GPT-4 on instruction-

following, truthfulness, honesty, and helpfulness, the standard source of preference pairs for DPO[14]. **OpenThoughts3** is 1.2M math, code, and science prompts with long reasoning traces from QwQ-32B, the result of more than a thousand controlled ablations on every step of the generation pipeline, and the set that pushed a 7B model to 53% on AIME 2025[15]. Each is a different kind of set — demonstrations, preferences, reasoning traces — and each has a different way of being bad.

**Small, hard, new — in that order.** The experiment that reset the field's intuition was LIMA: 1,000 carefully written demonstrations, no RLHF, and a 65B model whose outputs were preferred or tied against GPT-4's 43% of the time[16]. Its "superficial alignment hypothesis" — a base model already contains the knowledge, and SFT mostly teaches format and which sub-distribution to use — explains why the selection results of §11.4 keep finding that 5–10% of a set is as good as the whole: the other 90% teaches what the model already does. For a demonstration set like UltraChat the audit is therefore: IFD under your base model, keep the hard half; Min-K% to drop anything the base model has already memorized (common with GPT-generated data, which is widely re-posted); then the §11.3 judge for correctness, because GPT-3.5-era dialogues contain confident errors that a 2026 base model will unlearn its way into.

**Source quality beats source diversity.** This is the counter-intuitive finding from the OpenThoughts ablations, and it generalizes. Mixing the top two question sources for code scored 41.3 on their average; mixing the top sixteen scored 36.4 — a five-point loss for "diversity" — and the gap between the best and worst single source was 17 points[15]. The same study found that LLM-based question filters (by estimated difficulty, or by the length of a strong model's response) beat fastText and embedding filters, that sampling multiple traces per question scaled the useful data by 16× or more, that a weaker-scoring teacher (QwQ-32B) was a better teacher than a stronger one (DeepSeek-R1), and — the one that surprises everyone — that none of the answer-verification and filtering methods they tried gave significant improvements[15]. Read that last result carefully before copying it: it says that at 1.2M examples with a good teacher, the wrong traces are rare and diluted enough not to matter for benchmark scores; it does not say that verification is useless for a 10k-example set, or for a domain where the teacher is weak.

**Preference data has its own failure modes.** A preference pair is a judgment, not a fact, and the judge (GPT-4, for UltraFeedback) carries the biases of §11.3 into the dataset. The audit for a preference set is three numbers: the *length ratio* of chosen to rejected responses (if chosen is systematically longer, DPO will learn verbosity; the mitigation is to length-balance pairs or to subtract a length term from the reward); the *agreement rate* between the dataset's judge and a human sample on a few hundred pairs, which is the $\kappa$ of §11.3 again; and the *margin distribution* — pairs whose scores differ by a fraction of a point are noise, and keeping only pairs with a clear margin (UltraFeedback's own "binarized" variant does this) trades size for signal. Figure 11.6 shows the three audits as the funnel every post-training set should go through before training.

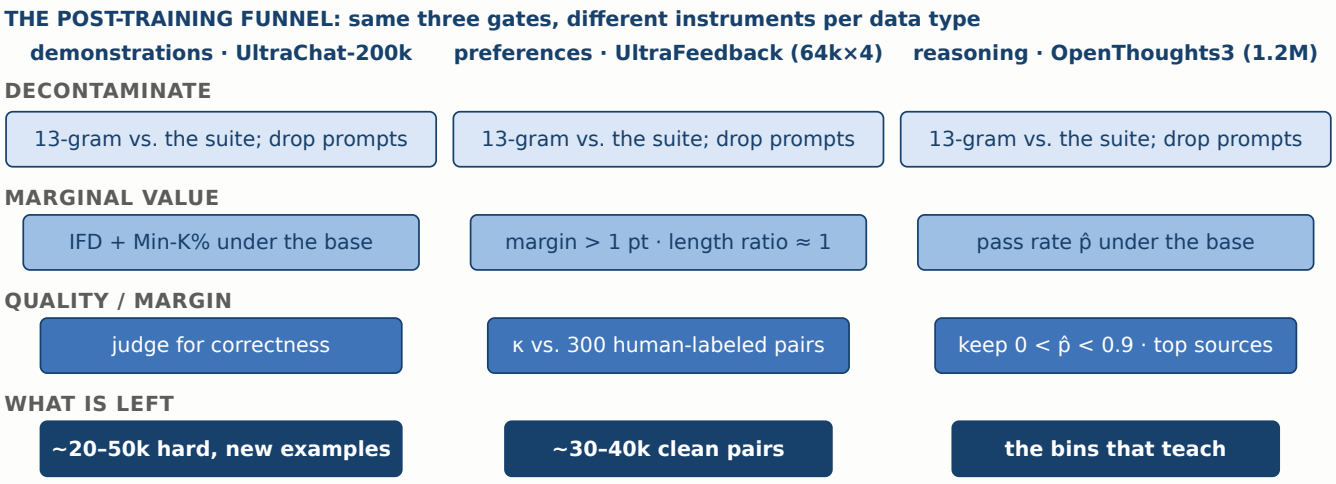



**Figure 11.6 — Three data types, one funnel.** Decontaminate first because it is cheap and non-negotiable; measure marginal value under *your* base model, not the one the dataset's authors used; audit quality or margin last, on what survived. What is left is smaller than what you downloaded, and better.

## 11.7 What the big labs do

The published recipes converge on the three gates, with the labs that release the most detail also being the ones that run the most audits. Table 11.1 collects what is disclosed; the pattern to notice is that every row has a held-out evaluation that the data process was not allowed to see.

**Table 11.1 — Data validation, as disclosed.** What each recipe reports about relevance, quality, and knowledge/contamination checks on its training data.

| RECIPE | RELEVANCE / SELECTION | QUALITY | KNOWLEDGE & CONTAMINATION |
|---|---|---|---|
| Apple BETR[1] | Embedding rank against 14k+ benchmark train items; fastText on the ranks; keep top 3–30% by compute | None beyond the pool's own filtering — relevance replaces quality | Separate target and held-out suites; reports the held-out drop; decontamination ablated |
| Llama 3[17] | Scaling-law experiments on small models to set the mix; classifier-selected high-quality data for annealing | fastText and RoBERTa quality classifiers; model-based code/math filters | Decontamination of pre-training data against evaluations; rejection sampling of SFT data with reward-model and verifier checks |
| Tülu 3[12] | Per-skill prompt curation; ablations on each source's contribution to the mix | Synthetic persona-driven generation with quality filtering; | Aggressive n-gram decontamination with a released toolkit; development vs. unseen |

| RECIPE | RELEVANCE / SELECTION | QUALITY | KNOWLEDGE & CONTAMINATION |
|---|---|---|---|
| | | rejection sampling against verifiers | suites; contaminated public sets dropped |
| OpenThoughts3[15] | Question sources ranked by downstream effect; top-2 sources beat top-16 | LLM difficulty / response-length filters beat fastText and embeddings; verification gave no significant gain at 1.2M | Decontaminated against evaluation sets; scaling curves on held-out AIME 2025 / LiveCodeBench |
| DeepSeek-R1[18] | Rejection sampling from the RL checkpoint: ~600k reasoning and ~200k non-reasoning examples for the final SFT | Rule-based verification for reasoning; generative reward model for the rest; readability filters | Benchmarks reported with contamination caveats; verifier-checkable rewards reduce the value of leaked answers |

## 11.8 Practical considerations

**Validate against the model you will train, at the scale you can afford.** Every measurement in §11.4 depends on the checkpoint. IFD and pass rate under a 1B proxy are a fair guide for a 7B target — the small-selects-for-large result holds across LESS, the perplexity reference, and DoReMi — but they are not the same numbers, and the marginal-value bins drift as training proceeds. Re-measure after each stage of Chapter 10's curriculum, and re-measure the pass-rate bins during RL.

**Budget the audits as a line item.** A thousand human labels (two annotators, two days), a judge pass over the survivors (a few hundred dollars of API or a few GPU-hours of an open judge), and a forward pass of your model over every benchmark item and every SFT candidate (hours, not days). Against a 66K-GPU-hour training run, the entire validation layer is under 1% of the cost and is the only part that can tell you the run measured something real.

**Keep the three scores in the lineage record.** Chapter 10's `LINEAGE` dictionary gains three keys per example — relevance rank, quality score with the threshold and the calibration table, membership score with the threshold text — and the dataset card gains a contamination section with the three tests of §11.5 per benchmark. When a number on the leaderboard is questioned a year from now, this is what answers it.

**For a Spanish-first model, re-derive every threshold.** Judges score Spanish lower than English for equal quality; embedding models place Spanish and English versions of the same question far apart, so a BETR-style ranking against English benchmark items under-selects Spanish documents unless the target set includes translated or native Spanish items (Global MMLU, the Latin American sets from Chapter 9); and Min-K% thresholds set on English post-cutoff text are wrong for Spanish, where the base model's log-probabilities are uniformly lower. Calibrate per language with the §11.3 sample, stratified by language.

## Common pitfalls

### Targeting the benchmark you report.

BETR's held-out drop is the measurement: selection toward a suite makes the suite stop measuring. Target a broad training-split set, report a suite you never targeted, and watch the gap.

### Trusting the judge's F1 from someone else's distribution.

The 0.82 was measured on FineWeb's English crawl. Your Spanish forum data, your code, your synthetic dialogues are a different distribution; the calibration sample is per distribution, not per model.

### Decontaminating by n-grams and calling it done.

Paraphrases, translations, and renumbered problems pass. Run the membership and completion tests on the trained model; they are the only ones that see what the training set looked like after everyone's rephrasing.

### Keeping everything the base model already knows.

Half of a downloaded SFT set may be at IFD ≈ 1 or pass rate ≈ 1 under your base. Those examples cost tokens and teach confidence, not capability.

### Confusing "verification did not help" with "verification does not matter."

The OpenThoughts result is at 1.2M examples with a strong teacher. At 10k examples, or with a teacher weak in your domain, wrong traces are a larger fraction and are not diluted away.

### Reading the mean when the tail is the signal.

Perplexity averages hide memorization; Min-K% exists because the tail is where membership shows. The same applies to preference margins: the pairs near zero margin are noise, not data.

## 11.9 Decision framework

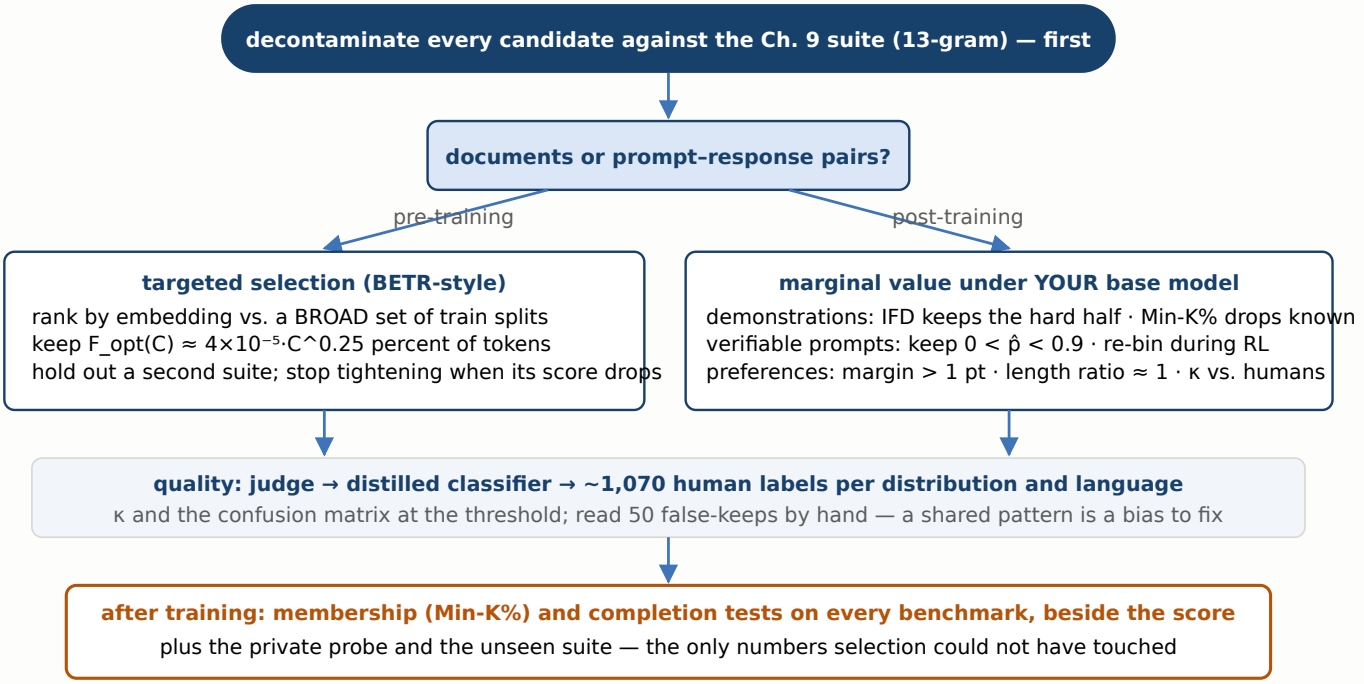



**Figure 11.7 — Validation in order.** Decontamination is first because it is cheap and irreversible once skipped; selection and marginal value depend on the data type; quality calibration is shared; and the contamination tests come last because they need the trained model. Every box writes to the lineage record.

**When it hurts.** Targeted selection hurts when the target suite is narrow (single-benchmark targeting lost 4–9 points on the joint average) and when the model is large enough to want breadth (above $10^{23}$ FLOPs the law says keep most of the pool). Marginal-value filtering hurts when the base model is weak in the domain — if nothing passes at $\hat{p} > 0$, the "bins that teach" are empty and you need the easier data you were about to drop, or an SFT stage before RL. Judge filtering hurts most quietly: a biased judge makes a homogeneous dataset that scores well on the judge's own distribution, which is why the human calibration sample is not optional.

**How this scales.** The validation layer costs about the same at every model size — a thousand labels, a judge pass over the survivors, a forward pass per item — while the training run it protects grows with $6ND$. At 1B it is a few percent of the budget; at 70B it is a rounding error; and the cost of *not* doing it — a run whose headline number turns out to be contamination — scales with the run. The retained fraction is the one knob that moves with scale, and BETR's law gives you its direction: loosen as you grow.

## 11.10 Summary

A dataset is validated when it answers three independent questions. Does it match the target? Measure relevance by ranking documents against the training splits of a broad benchmark suite, keep a fraction that grows with compute, and watch a held-out suite for the moment the filter starts to Goodhart — BETR's 2× compute multiplier and its third-place finish on held-out tasks are two sides of one mechanism. Is it good? Use a judge, distill it into a classifier, and calibrate both against about a thousand human labels per distribution and language, reading the false-keep cell by hand. Does the

model already know it? Measure it with per-token log-probabilities — Min-K% for membership, IFD for instruction data, pass rate for verifiable prompts — and keep what is hard and new, because the published selection results agree that 5–10% of a typical set carries its value. Turned toward benchmarks, the same instruments find contamination that n-gram overlap cannot: GSM1k's 8-point drops, the generation-probability correlation, and the leaderboard dynamics of 2025 are the evidence that a benchmark you optimized for has stopped being a benchmark. Run the membership and completion tests on the trained model, report them beside the score, and keep an unseen suite and a private probe that the data process was never allowed to see. Chapter 27 will generate data; this chapter is how you will know whether to keep it.

---

## References


1. Larsen, A. B. L., Gunter, T., et al. (2025). Language Models Improve When Pretraining Data Matches Target Tasks (BETR). arXiv:2507.12466. 500+ models from $10^{19}$ to $10^{22}$ FLOPs; 2.1× compute multiplier over DCLM-Baseline and 4.7× over unfiltered; 9/10 core tasks; F_opt(C) ≈ $4\times10^{-5}\cdot$C^0.25; held-out drop (44.2 vs. 46.3); single-benchmark targeting 53.5–58.7 vs. 62.3 joint; two-step design 62.2 vs. 59.7; decontamination ablation 62.5 vs. 62.3.
2. Xia, M., Malladi, S., Gururangan, S., Arora, S., & Chen, D. (2024). LESS: Selecting Influential Data for Targeted Instruction Tuning. *ICML*. arXiv:2402.04333. Projected LoRA-gradient datastore; Adam-aware influence; 5% selected data often beats the full set; small models select for larger ones across families.
3. Penedo, G., et al. (2024). The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. arXiv:2406.17557. Llama-3-70B-Instruct educational rubric; 450k judged documents; distilled classifier with F1 0.82 at threshold 3.
4. Sachdeva, N., et al. (2024). How to Train Data-Efficient LLMs. arXiv:2402.09668. Ask-LLM and density sampling among 19 samplers; rejecting 90% of data still beats full-data training; up to 70% faster convergence.
5. Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*. arXiv:2306.05685. GPT-4 agreement with humans above 80%, matching human–human agreement; position, verbosity, and self-enhancement biases.
6. Shi, W., et al. (2024). Detecting Pretraining Data from Large Language Models (Min-K% Prob). *ICLR*. arXiv:2310.16789. Mean of the k% lowest token log-probabilities; WikiMIA benchmark; +7.4 over prior methods; applied to copyrighted books and contaminated benchmark items.
7. Li, M., et al. (2024). From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning (IFD). *NAACL*. arXiv:2308.12032. Instruction-Following Difficulty ratio; 5% of Alpaca and 10% of WizardLM match full-data training.
8. Liu, W., et al. (2024). What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning (Deita). *ICLR*. arXiv:2312.15685. Complexity, quality, diversity scoring; 6K SFT examples match models trained on 10× more; MT-Bench 7.55 with 10K DPO pairs.
9. Yu, Q., et al. (2025). DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476. Dynamic sampling: prompts with all-correct or all-wrong groups carry zero advantage and are filtered out.

10. Zhang, H., et al. (2024). A Careful Examination of Large Language Model Performance on Grade School Arithmetic (GSM1k). arXiv:2405.00332. Up to 8-point drops from GSM8K to GSM1k; systematic overfitting in several families; Spearman $r^2 = 0.36$ between generation probability of GSM8K items and the gap.

11. Singh, S., et al. (2025). The Leaderboard Illusion. arXiv:2504.20879. 27 private variants from one provider before release; ~19–20% of Arena data each to the top two providers vs. ~30% to 83 open-weight models; up to 112% relative gains from additional Arena data.

12. Lambert, N., et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. Development vs. unseen evaluation suites; aggressive decontamination with a released toolkit; the SFT mixture (allenai/tulu-3-sft-mixture, ~940k prompts).

13. Golchin, S., & Surdeanu, M. (2024). Time Travel in LLMs: Tracing Data Contamination in Large Language Models. *ICLR*. arXiv:2308.08493. Guided instruction with the dataset name and split; completion overlap measured against unguided completions.

14. Cui, G., et al. (2024). UltraFeedback: Boosting Language Models with Scaled AI Feedback. *ICML*. arXiv:2310.01377. 64k prompts × 4 responses, GPT-4 scores on four aspects; the standard DPO preference source. Ding, N., et al. (2023). Enhancing Chat Language Models by Scaling High-quality Instructional Conversations (UltraChat). arXiv:2305.14233.

15. Guha, E., et al. (2025). OpenThoughts: Data Recipes for Reasoning Models. arXiv:2506.04178. 1,000+ ablations; 1.2M examples with QwQ-32B as teacher; OpenThinker3-7B at 53% AIME 2025, 51% LiveCodeBench, 54% GPQA Diamond; top-2 sources 41.3 vs. top-16 36.4; verification without significant gains.

16. Zhou, C., et al. (2023). LIMA: Less Is More for Alignment. *NeurIPS*. arXiv:2305.11206. 1,000 curated demonstrations; outputs equivalent or preferred to GPT-4's in 43% of cases; the superficial alignment hypothesis.

17. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Quality classifiers, scaling-law mix selection, annealing data, decontamination, and rejection-sampled post-training data.

18. DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. Rejection sampling of ~600k reasoning and ~200k non-reasoning SFT examples; rule-based and generative reward models.